Volume 06 Specifications - Integration Notification Automation Analytics Specifications

Version v34 draft

README.md

Integration Notification Automation Analytics Specifications

Version: v34 draft

Status: Draft

Document ID: MAL-V06-V34-README

Purpose

This release folder contains the official v34 documentation set for Integration Notification Automation Analytics Specifications.

Scope

- Covers Integration.
- Covers Notification.
- Covers Automation.
- Covers Analytics.

Acceptance

- All documents contain implementation-level guidance.
- The release PDF and ZIP are generated and verified.
- MANIFEST and RELEASE_NOTES are updated for v34.

Integration Notification Automation Analytics Specifications Index

- spec-06-34-01-integration.md - Integration
- spec-06-34-02-notification.md - Notification
- spec-06-34-03-automation.md - Automation
- spec-06-34-04-analytics.md - Analytics

spec-06-34-01-integration.md

Integration Specification

Title: Integration Specification

Version: v34 draft

Status: Draft

Specification ID: MAL-V06-INTEGRATION-SPEC-01

Related Blueprint Documents

- Volume 03 Master Blueprint
- Volume 04 Canonical Dictionary
- Volume 05 Engineering Standards
- Prior Volume 06 Specifications

Purpose

Define implementation-level requirements for Integration in Mariam AI Enterprise OS so engineering teams can build, test, operate, and govern it consistently.

Scope

- Covers system responsibilities, runtime behavior, interfaces, events, storage, security, observability, and acceptance evidence.
- Includes interactions with Enterprise Core, AI Society, Runtime Ecosystem, Knowledge System, Capability System, Governance, and Infrastructure.
- Excludes application-specific code, vendor credentials, and temporary implementation shortcuts.

Responsibilities

- Maintain authoritative state and lifecycle rules for Integration.
- Validate inputs, permissions, dependencies, and policy constraints before execution.
- Emit auditable events for lifecycle transitions, failures, recovery, and acceptance.
- Provide deterministic behavior that can be tested and traced across releases.

Functional Requirements

- SHALL expose versioned public interfaces for create, read, update, execute, validate, and audit operations.
- SHALL validate all inbound requests against schema, identity, policy, and dependency rules.
- SHALL support idempotency keys for commands that may be retried.
- SHALL publish events for accepted, rejected, started, completed, failed, recovered, and archived states.
- SHALL produce evidence bundles for review, compliance, and release acceptance.

Non-Functional Requirements

- MUST preserve tenant isolation, least privilege, and traceable authorization.
- MUST support observable latency, throughput, failure rate, queue depth, and recovery metrics.

- MUST recover safely after process restart, duplicate event delivery, or partial storage failure.
- SHOULD degrade gracefully when optional providers, plugins, connectors, or models are unavailable.

Inputs

- User, agent, workflow, runtime, provider, plugin, connector, and governance requests.
- Configuration, policies, feature flags, schemas, secrets references, and dependency metadata.
- Runtime events, task graph state, knowledge records, capability scores, and audit context.

Outputs

- Versioned records, execution results, validation reports, events, metrics, traces, logs, and acceptance evidence.
- Human-readable status summaries for dashboards, operations, release notes, and incident review.

Public Interfaces

- create_integration(request)
- get_integration_state(id)
- validate_integration(payload)
- execute_integration(command)
- audit_integration(scope)

Internal Interfaces

- Event Bus for durable event publication and subscription.
- Service Container for dependency injection and lifecycle management.
- Runtime Manager for execution dispatch and cancellation.
- Storage layer for records, snapshots, indexes, and evidence.
- Governance service for approval, policy, risk, and compliance checks.

Events

- integration.created
- integration.validated
- integration.started
- integration.completed
- integration.failed
- integration.recovered
- integration.accepted

Storage Requirements

- Store canonical records with identifiers, owner, version, state, timestamps, policy version, and trace ID.
- Store immutable event history and mutable read models separately where practical.
- Store evidence, validation reports, and recovery metadata with retention rules.

Security Requirements

- Enforce authentication, authorization, tenant isolation, and field-level redaction.
- Protect secrets by reference only; never persist raw credentials in logs, events, Markdown, or exports.
- Require approval for privileged, destructive, externally visible, financial, or compliance-sensitive actions.

Observability Requirements

- Emit structured logs with correlation IDs and decision reasons.
- Emit metrics for request count, success rate, failure rate, latency, retries, recovery success, and acceptance failures.
- Provide traces across public interface, policy validation, runtime execution, storage, events, and callbacks.

Failure Modes

- Invalid input, missing dependency, expired permission, unavailable runtime, conflicting state, duplicate command, or failed downstream provider.
- Incomplete event delivery, partial persistence, timeout, stalled execution, or acceptance failure.

Recovery Rules

- Retry idempotent commands with bounded backoff.
- Rebuild read models from event history when projections drift.
- Escalate to governance when automatic recovery changes risk, scope, budget, deadline, or external side effects.
- Preserve failed attempts and recovery decisions as audit evidence.

Test Requirements

- Unit tests for schemas, state transitions, permission checks, and event payloads.
- Integration tests for public interfaces, storage, event bus, runtime dependencies, and governance approvals.
- Failure tests for retries, duplicate events, unavailable dependencies, and recovery paths.
- Acceptance tests proving traceability from requirement to evidence.

Acceptance Criteria

- The specification is complete enough to create implementation tickets without hidden architectural decisions.
- Interfaces, events, storage, security, observability, failures, recovery, and tests are documented.
- The document traces to Blueprint, Standards, and release artifacts.

Implementation Notes

- Prefer small versioned interfaces and append-only event history.
- Keep provider-specific details behind adapters.
- Treat auditability and recovery as first-class design constraints.
- Validate schemas before work reaches runtime execution.

Traceability

Source	Relationship
MAL-V06-INTEGRATION-SPEC-01	Defines v34 implementation requirements for Integration.

Source	Relationship
Volume 03	Blueprint source.
Volume 05	Engineering standards source.
RELEASE_NOTES_v34_draft.md	Release evidence.

Version History

Version	Date	Status	Notes
v34 draft	2026-06-29	Draft	Initial specification for Integration.

spec-06-34-02-notification.md

Notification Specification

Title: Notification Specification

Version: v34 draft

Status: Draft

Specification ID: MAL-V06-NOTIFICATION-SPEC-02

Related Blueprint Documents

- Volume 03 Master Blueprint
- Volume 04 Canonical Dictionary
- Volume 05 Engineering Standards
- Prior Volume 06 Specifications

Purpose

Define implementation-level requirements for Notification in Mariam AI Enterprise OS so engineering teams can build, test, operate, and govern it consistently.

Scope

- Covers system responsibilities, runtime behavior, interfaces, events, storage, security, observability, and acceptance evidence.
- Includes interactions with Enterprise Core, AI Society, Runtime Ecosystem, Knowledge System, Capability System, Governance, and Infrastructure.
- Excludes application-specific code, vendor credentials, and temporary implementation shortcuts.

Responsibilities

- Maintain authoritative state and lifecycle rules for Notification.
- Validate inputs, permissions, dependencies, and policy constraints before execution.
- Emit auditable events for lifecycle transitions, failures, recovery, and acceptance.
- Provide deterministic behavior that can be tested and traced across releases.

Functional Requirements

- SHALL expose versioned public interfaces for create, read, update, execute, validate, and audit operations.
- SHALL validate all inbound requests against schema, identity, policy, and dependency rules.
- SHALL support idempotency keys for commands that may be retried.
- SHALL publish events for accepted, rejected, started, completed, failed, recovered, and archived states.
- SHALL produce evidence bundles for review, compliance, and release acceptance.

Non-Functional Requirements

- MUST preserve tenant isolation, least privilege, and traceable authorization.
- MUST support observable latency, throughput, failure rate, queue depth, and recovery metrics.

- MUST recover safely after process restart, duplicate event delivery, or partial storage failure.
- SHOULD degrade gracefully when optional providers, plugins, connectors, or models are unavailable.

Inputs

- User, agent, workflow, runtime, provider, plugin, connector, and governance requests.
- Configuration, policies, feature flags, schemas, secrets references, and dependency metadata.
- Runtime events, task graph state, knowledge records, capability scores, and audit context.

Outputs

- Versioned records, execution results, validation reports, events, metrics, traces, logs, and acceptance evidence.
- Human-readable status summaries for dashboards, operations, release notes, and incident review.

Public Interfaces

- create_notification(request)
- get_notification_state(id)
- validate_notification(payload)
- execute_notification(command)
- audit_notification(scope)

Internal Interfaces

- Event Bus for durable event publication and subscription.
- Service Container for dependency injection and lifecycle management.
- Runtime Manager for execution dispatch and cancellation.
- Storage layer for records, snapshots, indexes, and evidence.
- Governance service for approval, policy, risk, and compliance checks.

Events

- notification.created
- notification.validated
- notification.started
- notification.completed
- notification.failed
- notification.recovered
- notification.accepted

Storage Requirements

- Store canonical records with identifiers, owner, version, state, timestamps, policy version, and trace ID.
- Store immutable event history and mutable read models separately where practical.
- Store evidence, validation reports, and recovery metadata with retention rules.

Security Requirements

- Enforce authentication, authorization, tenant isolation, and field-level redaction.
- Protect secrets by reference only; never persist raw credentials in logs, events, Markdown, or exports.
- Require approval for privileged, destructive, externally visible, financial, or compliance-sensitive actions.

Observability Requirements

- Emit structured logs with correlation IDs and decision reasons.
- Emit metrics for request count, success rate, failure rate, latency, retries, recovery success, and acceptance failures.
- Provide traces across public interface, policy validation, runtime execution, storage, events, and callbacks.

Failure Modes

- Invalid input, missing dependency, expired permission, unavailable runtime, conflicting state, duplicate command, or failed downstream provider.
- Incomplete event delivery, partial persistence, timeout, stalled execution, or acceptance failure.

Recovery Rules

- Retry idempotent commands with bounded backoff.
- Rebuild read models from event history when projections drift.
- Escalate to governance when automatic recovery changes risk, scope, budget, deadline, or external side effects.
- Preserve failed attempts and recovery decisions as audit evidence.

Test Requirements

- Unit tests for schemas, state transitions, permission checks, and event payloads.
- Integration tests for public interfaces, storage, event bus, runtime dependencies, and governance approvals.
- Failure tests for retries, duplicate events, unavailable dependencies, and recovery paths.
- Acceptance tests proving traceability from requirement to evidence.

Acceptance Criteria

- The specification is complete enough to create implementation tickets without hidden architectural decisions.
- Interfaces, events, storage, security, observability, failures, recovery, and tests are documented.
- The document traces to Blueprint, Standards, and release artifacts.

Implementation Notes

- Prefer small versioned interfaces and append-only event history.
- Keep provider-specific details behind adapters.
- Treat auditability and recovery as first-class design constraints.
- Validate schemas before work reaches runtime execution.

Traceability

Source	Relationship
MAL-V06-NOTIFICATION-SPEC-02	Defines v34 implementation requirements for Notification.

Source	Relationship
Volume 03	Blueprint source.
Volume 05	Engineering standards source.
RELEASE_NOTES_v34_draft.md	Release evidence.

Version History

Version	Date	Status	Notes
v34 draft	2026-06-29	Draft	Initial specification for Notification.

spec-06-34-03-automation.md

Automation Specification

Title: Automation Specification

Version: v34 draft

Status: Draft

Specification ID: MAL-V06-AUTOMATION-SPEC-03

Related Blueprint Documents

- Volume 03 Master Blueprint
- Volume 04 Canonical Dictionary
- Volume 05 Engineering Standards
- Prior Volume 06 Specifications

Purpose

Define implementation-level requirements for Automation in Mariam AI Enterprise OS so engineering teams can build, test, operate, and govern it consistently.

Scope

- Covers system responsibilities, runtime behavior, interfaces, events, storage, security, observability, and acceptance evidence.
- Includes interactions with Enterprise Core, AI Society, Runtime Ecosystem, Knowledge System, Capability System, Governance, and Infrastructure.
- Excludes application-specific code, vendor credentials, and temporary implementation shortcuts.

Responsibilities

- Maintain authoritative state and lifecycle rules for Automation.
- Validate inputs, permissions, dependencies, and policy constraints before execution.
- Emit auditable events for lifecycle transitions, failures, recovery, and acceptance.
- Provide deterministic behavior that can be tested and traced across releases.

Functional Requirements

- SHALL expose versioned public interfaces for create, read, update, execute, validate, and audit operations.
- SHALL validate all inbound requests against schema, identity, policy, and dependency rules.
- SHALL support idempotency keys for commands that may be retried.
- SHALL publish events for accepted, rejected, started, completed, failed, recovered, and archived states.
- SHALL produce evidence bundles for review, compliance, and release acceptance.

Non-Functional Requirements

- MUST preserve tenant isolation, least privilege, and traceable authorization.
- MUST support observable latency, throughput, failure rate, queue depth, and recovery metrics.

- MUST recover safely after process restart, duplicate event delivery, or partial storage failure.
- SHOULD degrade gracefully when optional providers, plugins, connectors, or models are unavailable.

Inputs

- User, agent, workflow, runtime, provider, plugin, connector, and governance requests.
- Configuration, policies, feature flags, schemas, secrets references, and dependency metadata.
- Runtime events, task graph state, knowledge records, capability scores, and audit context.

Outputs

- Versioned records, execution results, validation reports, events, metrics, traces, logs, and acceptance evidence.
- Human-readable status summaries for dashboards, operations, release notes, and incident review.

Public Interfaces

- create_automation(request)
- get_automation_state(id)
- validate_automation(payload)
- execute_automation(command)
- audit_automation(scope)

Internal Interfaces

- Event Bus for durable event publication and subscription.
- Service Container for dependency injection and lifecycle management.
- Runtime Manager for execution dispatch and cancellation.
- Storage layer for records, snapshots, indexes, and evidence.
- Governance service for approval, policy, risk, and compliance checks.

Events

- automation.created
- automation.validated
- automation.started
- automation.completed
- automation.failed
- automation.recovered
- automation.accepted

Storage Requirements

- Store canonical records with identifiers, owner, version, state, timestamps, policy version, and trace ID.
- Store immutable event history and mutable read models separately where practical.
- Store evidence, validation reports, and recovery metadata with retention rules.

Security Requirements

- Enforce authentication, authorization, tenant isolation, and field-level redaction.
- Protect secrets by reference only; never persist raw credentials in logs, events, Markdown, or exports.
- Require approval for privileged, destructive, externally visible, financial, or compliance-sensitive actions.

Observability Requirements

- Emit structured logs with correlation IDs and decision reasons.
- Emit metrics for request count, success rate, failure rate, latency, retries, recovery success, and acceptance failures.
- Provide traces across public interface, policy validation, runtime execution, storage, events, and callbacks.

Failure Modes

- Invalid input, missing dependency, expired permission, unavailable runtime, conflicting state, duplicate command, or failed downstream provider.
- Incomplete event delivery, partial persistence, timeout, stalled execution, or acceptance failure.

Recovery Rules

- Retry idempotent commands with bounded backoff.
- Rebuild read models from event history when projections drift.
- Escalate to governance when automatic recovery changes risk, scope, budget, deadline, or external side effects.
- Preserve failed attempts and recovery decisions as audit evidence.

Test Requirements

- Unit tests for schemas, state transitions, permission checks, and event payloads.
- Integration tests for public interfaces, storage, event bus, runtime dependencies, and governance approvals.
- Failure tests for retries, duplicate events, unavailable dependencies, and recovery paths.
- Acceptance tests proving traceability from requirement to evidence.

Acceptance Criteria

- The specification is complete enough to create implementation tickets without hidden architectural decisions.
- Interfaces, events, storage, security, observability, failures, recovery, and tests are documented.
- The document traces to Blueprint, Standards, and release artifacts.

Implementation Notes

- Prefer small versioned interfaces and append-only event history.
- Keep provider-specific details behind adapters.
- Treat auditability and recovery as first-class design constraints.
- Validate schemas before work reaches runtime execution.

Traceability

Source	Relationship
MAL-V06-AUTOMATION-SPEC-03	Defines v34 implementation requirements for Automation.

Source	Relationship
Volume 03	Blueprint source.
Volume 05	Engineering standards source.
RELEASE_NOTES_v34_draft.md	Release evidence.

Version History

Version	Date	Status	Notes
v34 draft	2026-06-29	Draft	Initial specification for Automation.

spec-06-34-04-analytics.md

Analytics Specification

Title: Analytics Specification

Version: v34 draft

Status: Draft

Specification ID: MAL-V06-ANALYTICS-SPEC-04

Related Blueprint Documents

- Volume 03 Master Blueprint
- Volume 04 Canonical Dictionary
- Volume 05 Engineering Standards
- Prior Volume 06 Specifications

Purpose

Define implementation-level requirements for Analytics in Mariam AI Enterprise OS so engineering teams can build, test, operate, and govern it consistently.

Scope

- Covers system responsibilities, runtime behavior, interfaces, events, storage, security, observability, and acceptance evidence.
- Includes interactions with Enterprise Core, AI Society, Runtime Ecosystem, Knowledge System, Capability System, Governance, and Infrastructure.
- Excludes application-specific code, vendor credentials, and temporary implementation shortcuts.

Responsibilities

- Maintain authoritative state and lifecycle rules for Analytics.
- Validate inputs, permissions, dependencies, and policy constraints before execution.
- Emit auditable events for lifecycle transitions, failures, recovery, and acceptance.
- Provide deterministic behavior that can be tested and traced across releases.

Functional Requirements

- SHALL expose versioned public interfaces for create, read, update, execute, validate, and audit operations.
- SHALL validate all inbound requests against schema, identity, policy, and dependency rules.
- SHALL support idempotency keys for commands that may be retried.
- SHALL publish events for accepted, rejected, started, completed, failed, recovered, and archived states.
- SHALL produce evidence bundles for review, compliance, and release acceptance.

Non-Functional Requirements

- MUST preserve tenant isolation, least privilege, and traceable authorization.
- MUST support observable latency, throughput, failure rate, queue depth, and recovery metrics.

- MUST recover safely after process restart, duplicate event delivery, or partial storage failure.
- SHOULD degrade gracefully when optional providers, plugins, connectors, or models are unavailable.

Inputs

- User, agent, workflow, runtime, provider, plugin, connector, and governance requests.
- Configuration, policies, feature flags, schemas, secrets references, and dependency metadata.
- Runtime events, task graph state, knowledge records, capability scores, and audit context.

Outputs

- Versioned records, execution results, validation reports, events, metrics, traces, logs, and acceptance evidence.
- Human-readable status summaries for dashboards, operations, release notes, and incident review.

Public Interfaces

- create_analytics(request)
- get_analytics_state(id)
- validate_analytics(payload)
- execute_analytics(command)
- audit_analytics(scope)

Internal Interfaces

- Event Bus for durable event publication and subscription.
- Service Container for dependency injection and lifecycle management.
- Runtime Manager for execution dispatch and cancellation.
- Storage layer for records, snapshots, indexes, and evidence.
- Governance service for approval, policy, risk, and compliance checks.

Events

- analytics.created
- analytics.validated
- analytics.started
- analytics.completed
- analytics.failed
- analytics.recovered
- analytics.accepted

Storage Requirements

- Store canonical records with identifiers, owner, version, state, timestamps, policy version, and trace ID.
- Store immutable event history and mutable read models separately where practical.
- Store evidence, validation reports, and recovery metadata with retention rules.

Security Requirements

- Enforce authentication, authorization, tenant isolation, and field-level redaction.
- Protect secrets by reference only; never persist raw credentials in logs, events, Markdown, or exports.
- Require approval for privileged, destructive, externally visible, financial, or compliance-sensitive actions.

Observability Requirements

- Emit structured logs with correlation IDs and decision reasons.
- Emit metrics for request count, success rate, failure rate, latency, retries, recovery success, and acceptance failures.
- Provide traces across public interface, policy validation, runtime execution, storage, events, and callbacks.

Failure Modes

- Invalid input, missing dependency, expired permission, unavailable runtime, conflicting state, duplicate command, or failed downstream provider.
- Incomplete event delivery, partial persistence, timeout, stalled execution, or acceptance failure.

Recovery Rules

- Retry idempotent commands with bounded backoff.
- Rebuild read models from event history when projections drift.
- Escalate to governance when automatic recovery changes risk, scope, budget, deadline, or external side effects.
- Preserve failed attempts and recovery decisions as audit evidence.

Test Requirements

- Unit tests for schemas, state transitions, permission checks, and event payloads.
- Integration tests for public interfaces, storage, event bus, runtime dependencies, and governance approvals.
- Failure tests for retries, duplicate events, unavailable dependencies, and recovery paths.
- Acceptance tests proving traceability from requirement to evidence.

Acceptance Criteria

- The specification is complete enough to create implementation tickets without hidden architectural decisions.
- Interfaces, events, storage, security, observability, failures, recovery, and tests are documented.
- The document traces to Blueprint, Standards, and release artifacts.

Implementation Notes

- Prefer small versioned interfaces and append-only event history.
- Keep provider-specific details behind adapters.
- Treat auditability and recovery as first-class design constraints.
- Validate schemas before work reaches runtime execution.

Traceability

Source	Relationship
MAL-V06-ANALYTICS-SPEC-04	Defines v34 implementation requirements for Analytics.

Source	Relationship
Volume 03	Blueprint source.
Volume 05	Engineering standards source.
RELEASE_NOTES_v34_draft.md	Release evidence.

Version History

Version	Date	Status	Notes
v34 draft	2026-06-29	Draft	Initial specification for Analytics.